

CM-2 Data cleaning and preprocessing

3TLDE.A13 - Législation Numérique et Enjeux de l'Analyse des Données



30 April 2026

From inspection to cleaning

- In CM-1, we learned how to **load**, **inspect**, and **summarize** a dataset.
- In CM-2, the goal is different: we want to make the data **usable**, **consistent**, and **ready for analysis or machine learning**.
- Real datasets are rarely perfect.
- Before training a model, we must ask:
 - ▶ Are some values missing?
 - ▶ Are some rows duplicated?
 - ▶ Are some values impossible or suspicious?
 - ▶ Are the data types correct?

Attention

A model does not know whether the data is wrong. It will learn from whatever we give it.

Why is this lecture important?

- Data cleaning is one of the most time-consuming parts of real data work.
- Good cleaning improves:
 - ▶ the quality of descriptive statistics,
 - ▶ the quality of visualizations,
 - ▶ the stability of machine learning models,
 - ▶ the reliability of conclusions.
- Bad cleaning can create new errors instead of fixing old ones.

Garbage in, garbage out

If the input data is unreliable, the output of the analysis or model will also be unreliable.

What can go wrong if we skip cleaning?

- A mean can be wrong because of a few impossible values.
- A category count can be wrong because Lyon and Lion are treated as different groups.
- A model can fail because a numerical column contains text.
- A duplicated row can make one case appear more important than it really is.
- Test results can look better than reality if preprocessing is done incorrectly.

Practical message

Cleaning is not cosmetic work. It changes the quality of the whole pipeline.

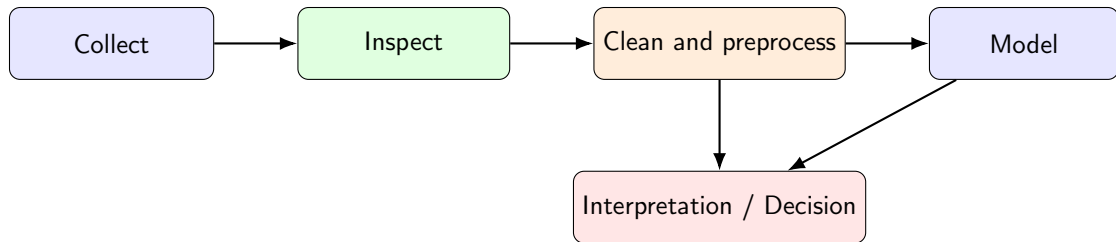
A simple example of dirty data

ID	Age	Salary	City	Purchased
1	24	2100	Lyon	Yes
2	35	unknown	Paris	No
3		3200	Lion	Yes
4	180	2800	Marseille	No
4	29	3100	Paris	Yes
5	29	3100	Paris	Yes

Problems in this table:

- missing age,
- non-numeric salary value,
- inconsistent city names,
- suspicious age,
- duplicated row.

Where cleaning appears in the workflow



- Cleaning comes after the first inspection.
- In practice, we often go back and forth between inspection and cleaning.

Learning objectives of CM-2

At the end of this lecture, you should be able to:

- explain why data cleaning is necessary;
- identify common data quality problems;
- detect and handle missing values;
- detect duplicated rows and decide whether to remove them;
- understand the basic idea of outlier detection with boxplots and IQR;
- correct data types and inconsistent formats;
- conduct standardization, normalization, and categorical encoding;
- understand why preprocessing must be done carefully before machine learning.

Table of contents

- 1 **Common data quality problems**
- 2 Data types and format consistency
- 3 Missing values
- 4 Duplicates and inconsistent records
- 5 Outliers and suspicious values
- 6 Preprocessing for machine learning
- 7 Summary

Typical data quality problems

- **Wrong data types:** a number is stored as text, or a date is stored as a plain string.
- **Inconsistent categories:** for example, Male, male, and M.
- **Invalid values:** negative age, impossible dates, impossible measurements.
- **Missing values:** some information is absent.
- **Duplicated rows:** the same record appears more than once.
- **Outliers:** some values are very far from the others.
- **Different scales:** one variable is in euros, another in thousands of euros.

Example of messy data sheet

	A	B	C	D	E	F	G	H	I
1	ID	Name	Gender	Age	Date of Birth	Country	Signup Date	Purchase Amount	Notes
2	1001	Alice Martin	Female	29	12/05/1995	France	2023-01-15	1,200.50 €	Good customer
3	1002	bob smith	male	34	1989-07-23	france	15/01/2023	€980	
4	1003	Charlie Brown	M		02/30/1990	USA	2023/02/30	-50 €	Refund
5	1004	Diana Prince	Female	-5	1992-11-31	UK	31-02-2023	1 000,00 €	
6	1005	Evan Lee	male	45	1978-03-10	Canada	2023-03-05	2,500.75 €	
7	1006	Fiona Gallagher	FEMALE	27	1997-06-18	USA	2023-03-07	15 €	
8	1007	George King	Male	120	1900-01-01	UK	2023-03-10	1.000.000,00 €	VIP??
9	1008	Hannah Scott	female	31	1993-08-25	Australia	not available	300	
10	1009	Ian Wright	M	abc	08/15/1991	US	2023-04-01	€1,1 k	
11	1010	Jane Doe		28	1996-09-05	France	2023-04-03	-10 €	Test order
12	1002	bob smith	male	34	1989-07-23	France	15/01/2023	€980	
13	1011	Zoe Adams	F	22	2001-13-01	Germany	2023-04-31	500000	

Table of contents

- 1 Common data quality problems
- 2 Data types and format consistency**
- 3 Missing values
- 4 Duplicates and inconsistent records
- 5 Outliers and suspicious values
- 6 Preprocessing for machine learning
- 7 Summary

Why data types matter?

- Data type determines what operations are possible.
- If a numerical column is stored as text, some calculations will fail.
- If a date is stored as a plain string, time-based analysis becomes difficult.

Typical problem

A column looks numerical to the human eye, but Python sees it as text.

Common types in tabular data

- **Integer**: number of counting.
- **Float**: age, salary, temperature.
- **String**: names, comments, codes.
- **Boolean**: yes/no, true/false.
- **Datetime**: dates and timestamps.
- **Category**: repeated labels with a limited set of values.

Strings that should be numbers

Examples:

- "24"
- "3.14"
- "1,200"
- "€2500"

Problem

These values may require cleaning before conversion because commas, currency symbols, or spaces can block numerical operations.

Converting a column to numeric: `to_numeric()`

```
1 df['Salary'] = pd.to_numeric(df['Salary'], errors='coerce')
```

- With `errors='coerce'`, invalid text becomes NaN.
- This is often useful because it transforms a hidden type problem into an explicit missing value problem.

Date formats can be inconsistent

Examples of the same date:

- 2026-05-04
- 04/05/2026
- May 4, 2026

Risk

A date like 04/05/2026 can mean 4 May or 5 April depending on the convention.

Converting a column to datetime: `to_datetime()`

```
1 df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

Benefit

Once the type is correct, we can sort by date, extract the year or month, and compute time differences.

Inconsistent categories

A categorical variable may contain several labels for the same concept:

- Male, male, M
- Yes, yes, Y
- Lyon, lyon, LYON

Consequence

Group counts become wrong because one category is split into several labels.

Simple text standardization

```
1 df['City'] = df['City'].str.strip().str.lower().str.title()
```

- `strip()` removes extra spaces.
- `lower()` converts text to lowercase.
- `title()` capitalizes the first letter of each word.

Result

" Lyon ", "LYON", and "lyon" all become "Lyon". "new york" will become "New York"

A small mapping example

After standardization, we sometimes still need explicit replacement:

- $m \rightarrow \text{male}$
- $f \rightarrow \text{female}$

```
1 gender_map = {  
2     'm': 'male',  
3     'f': 'female'  
4 }  
5  
6 df['Gender'] = df['Gender'].str.strip().str.lower().replace(gender_map)
```

Practical advice

Create a clear mapping dictionary when categories have several variants.

Table of contents

- 1 Common data quality problems
- 2 Data types and format consistency
- 3 Missing values**
- 4 Duplicates and inconsistent records
- 5 Outliers and suspicious values
- 6 Preprocessing for machine learning
- 7 Summary

What is a missing value?

- A missing value means that information is not available for a given cell.
- In Python and pandas, missing values often appear as:
 - ▶ NaN
 - ▶ None
 - ▶ empty cells
- In raw files, missing values may also appear as:
 - ▶ "?"
 - ▶ "unknown"
 - ▶ "NA"
 - ▶ special codes such as -9999999

Where do missing values come from?

- A user did not answer a question.
- A sensor failed during measurement.
- A value was not recorded.
- Two tables were merged, but some rows did not match.
- Some variables are not applicable for all individuals.

Important point

Missing data is common.

The question is not whether it exists, but how much of it exists and what to do with it.

Detecting missing values in pandas: `isna()` or `isnull()`

- We often count missing values **by column**.
- We can also compute the percentage of missing values.

```
1 import pandas as pd
2
3 missing_count = df.isna().sum(axis=0)
4 missing_ratio = df.isna().mean(axis=0) * 100
5
6 print(missing_count)
7 print(missing_ratio)
```


Treatment method 1: deletion

- **Row deletion:** remove rows containing missing values.
- **Column deletion:** remove columns with too many missing values.

Advantages

- simple,
- fast,
- no invented fake values.

Limitations

- loss of information,
- smaller dataset,
- possible bias.

Deleting missing values in pandas: **dropna()**

```
1 # remove rows with at least one missing value
2 clean_df = df.dropna()
3
4 # remove columns with at least one missing value
5 clean_df = df.dropna(axis=1)
6
7 # remove columns with more than 50% missing values
8 clean_df = df.loc[:, df.isna().mean() < 0.5]
```

Use with care

Deleting rows is acceptable only when we are sure that we are not losing too much useful information.

When is deletion acceptable?

- The dataset is large enough.
- The number of missing values is small.
- The missing values appear randomly.
- The removed variable is not important for the task.

Example

If 40% of a column is missing and the column is very important, deleting it may be a bad decision.

Treatment method 2: imputation for numerical variables

- **Mean imputation:** replace missing values by the column average.
- **Median imputation:** replace missing values by the column median.

Mean

Useful when the distribution is reasonably symmetric and without large outliers.

Median

More robust when the distribution is skewed or contains extreme values.

Example: mean vs median

Suppose we have salaries:

2000, 2100, 2200, 2300, 10000

- Mean = 3720
- Median = 2200

Interpretation

The mean is strongly affected by the extreme value 10000.

In this case, the median is often a better replacement for a missing salary.

Imputing numerical values in pandas: `mean()`, `median()` + `fillna()`

```
1 # mean imputation
2 df['Age'] = df['Age'].fillna(df['Age'].mean())
3
4 # median imputation
5 df['Salary'] = df['Salary'].fillna(df['Salary'].median())
```

Reminder

Imputation creates artificial values. It is useful, but it should not be done without thinking.

Treatment method 3: imputation for categorical variables

- For a categorical column, a common solution is to use the **mode**: the most frequent category.
- Example:
 - ▶ city values: Paris, Lyon, Paris, Marseille, missing
 - ▶ mode = Paris

Limitation

This may reinforce the majority class and hide useful information.

Imputing categorical values in pandas: `mode()` + `fillna()`

```
1 mode_city = df['City'].mode()[0]
2
3 df['City'] = df['City'].fillna(mode_city)
```

Alternative

Sometimes we create a new category, such as Unknown instead of filling with the mode.

When should we avoid naive imputation?

- When too many values are missing.
- When the missingness itself is meaningful.
- When filling with one constant distorts the distribution.
- When the variable is sensitive and important for later interpretation.

Example

If income is missing mainly for rich individuals, replacing all missing values with the mean can create a misleading result (a drifted distribution).

Missingness is not always random

- Missing values do not always have the same meaning.
- The **reason why a value is missing** can influence the analysis.

Three common situations

- **MCAR – Missing Completely At Random**

The value is missing by pure chance.

Example: a technical error deletes some cells randomly.

- **MAR – Missing At Random**

The missingness can be explained by another **observed** variable.

Example: younger people are less likely to report their income, but age is known.

- **MNAR – Missing Not At Random**

The missingness depends on the missing value itself or on an **unobserved** factor.

Example: people with very high income are less likely to report their income.

Key message

Missing values are not just “empty cells”. Before cleaning the data, we should ask: **why is this value missing?**

Table of contents

- 1 Common data quality problems
- 2 Data types and format consistency
- 3 Missing values
- 4 Duplicates and inconsistent records**
- 5 Outliers and suspicious values
- 6 Preprocessing for machine learning
- 7 Summary

What is a duplicate?

- A duplicate is a repeated record in the dataset.
- It may be:
 - ▶ a full duplicate: every column is identical;
 - ▶ a partial duplicate: only key columns are identical.

Example

Two rows with the same customer identifier, the same date, and the same purchase amount may represent the same event entered twice.

Why duplicates are a problem

- They change counts and percentages.
- They can bias the mean and other summary statistics.
- They can make a model overemphasize a repeated case.
- They can create the illusion that the dataset is larger than it really is.

Not every repeated entity is a duplicate

True duplicate

The same transaction is stored twice by mistake.

Not a duplicate

The same customer appears many times because the customer made many purchases.

Before removing duplicates, define clearly what one row represents: a person, a transaction, a measurement, or an event.

Detecting and removing duplicates in pandas: `uplicated()` and `drop_duplicates()`

```
1 # check duplicated rows
2 # Mark duplicates as True except for the first occurrence
3 dup = df.duplicated(keep='first')
4 print(dup.sum())
5
6 # show duplicated rows
7 print(df[df.duplicated()])
8
9 # remove duplicated rows
10 df = df.drop_duplicates()
```

Checking duplicates on key columns

```
1 # duplicates based only on selected columns
2 mask = df.duplicated(subset=['customer_id', 'date'])
3
4 print(df[mask])
```

Why this matters

Sometimes the full rows differ slightly,
but the key columns show that they refer to the same object or event.

Table of contents

- 1 Common data quality problems
- 2 Data types and format consistency
- 3 Missing values
- 4 Duplicates and inconsistent records
- 5 Outliers and suspicious values**
- 6 Preprocessing for machine learning
- 7 Summary

What is an outlier?

- An outlier is a value that is **much larger or much smaller** than most other values.
- Outliers are not always errors.
- They may come from:
 - ▶ data entry errors,
 - ▶ measurement errors,
 - ▶ rare but real cases.

Examples of suspicious values

- Age = 180 in a human population dataset.
- Exam score = 25 when the maximum should be 20.
- Temperature = -300 degrees Celsius.
- Negative number of counting.

Important distinction

A suspicious value is not automatically an outlier, and an outlier is not automatically an error.

Why outliers matter

- They can strongly affect the mean.
- They can increase the standard deviation.
- They can distort plots and make patterns harder to see.
- They can affect the performance of some machine learning algorithms.

Simple ways to detect outliers

- Sort the values and inspect the extremes.
- Look at summary statistics.
- Draw a histogram.
- Draw a scatter plot.
- Draw a boxplot.

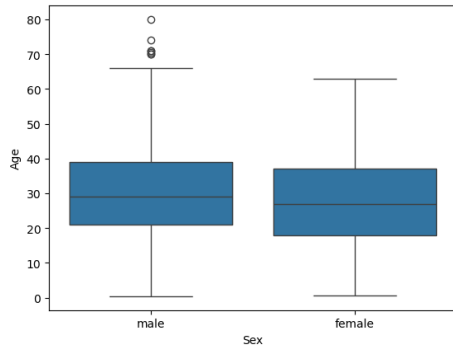
In this lecture, we focus on the **boxplot and the IQR rule** because they are simple and useful.

The basic idea of a boxplot

A boxplot summarizes a numerical distribution using:

- the first quartile Q_1 ,
- the median,
- the third quartile Q_3 ,
- possible outliers outside the usual range.

```
1 import seaborn as sns
2
3 sns.boxplot(data=df, y='col-of-interest')
```



Reading the plot

The box shows the central part of the data. Points far away from the box are potential outliers.

Interquartile range (IQR)

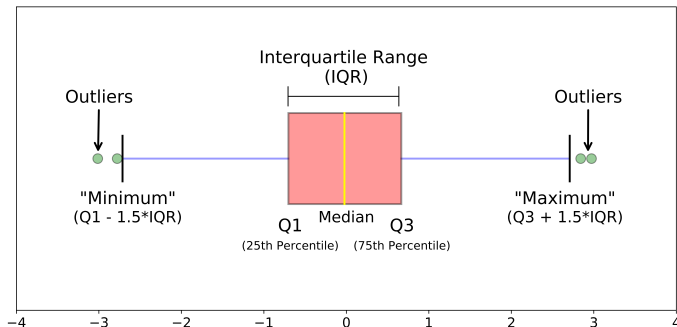
The interquartile range is defined as: $IQR = Q_3 - Q_1$

We often define potential outliers using:

$$\text{Lower bound} = Q_1 - 1.5 \times IQR$$

$$\text{Upper bound} = Q_3 + 1.5 \times IQR$$

Values outside this interval are considered **potential outliers**.



An example of IQR

Consider the values:

2, 3, 4, 5, 6, 7, 8, 30

- $Q_1 \approx 3.5$
- $Q_3 \approx 7.5$
- $IQR = 4$
- Upper bound $= 7.5 + 1.5 \times 4 = 13.5$

Conclusion

The value 30 is above 13.5, so it is a potential outlier.

Computing IQR in pandas

```
1 q1 = df['col-of-intrest'].quantile(0.25)
2 q3 = df['col-of-intrest'].quantile(0.75)
3 iqr = q3 - q1
4
5 lower = q1 - 1.5 * iqr
6 upper = q3 + 1.5 * iqr
7
8 outliers = df[(df['col-of-intrest'] < lower) |
9               (df['col-of-intrest'] > upper)]
```

What can we do after detecting outliers?

- **Keep them** if they are real and important.
- **Remove them** if they are clear errors.
- **Cap them** by limiting extreme values.
- **Investigate them** before making a decision.

Important

Do not remove outliers automatically. First ask why they are there. Outliers are sometimes valuable.

- In fraud detection, unusual transactions may be exactly what we want to find.
- In medicine, rare cases may deserve special attention.
- In industrial monitoring, abnormal sensor values may indicate a fault.

Table of contents

- 1 Common data quality problems
- 2 Data types and format consistency
- 3 Missing values
- 4 Duplicates and inconsistent records
- 5 Outliers and suspicious values
- 6 Preprocessing for machine learning**
- 7 Summary

Cleaning vs preprocessing

Cleaning

Fix problems in the data:

- wrong types,
- inconsistent labels.
- missing values,
- duplicates,
- outliers

Preprocessing

Transform the data for the model:

- encoding,
- scaling,
- train/test split,
- feature preparation.

Why most models need preprocessing?

- Many models accept only numerical input.
- Features may have very different scales.
- Some variables contain categories, not numbers.
- Training and evaluation must be separated correctly.

Feature scaling: the main idea

Consider two features:

- Age: values between 18 and 70
- Income: values between 1000 and 100000

Problem

Without scaling, the larger-scale variable may dominate distance-based or gradient-based methods.

Standardization

Standardization transforms a variable using:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

- The transformed variable has mean close to 0.
- Its standard deviation is close to 1.

Normalization (min-max scaling)

A common normalization formula is:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Values are rescaled to the interval $[0, 1]$.
- This is useful when we want all features in the same bounded range.

Standardization or normalization?

Standardization

Often used for:

- linear models,
- logistic regression,
- neural networks,
- PCA.

Normalization

Often useful when:

- a bounded range is needed,
- input scales must be comparable,
- distances are sensitive to scale.

Why categorical variables need encoding

A model usually cannot work directly with text values such as:

- red, blue, green
- Paris, Lyon, Marseille

Need

We must convert categories into numerical representations.

Label encoding

Example:

- low $\rightarrow$ 0
- medium $\rightarrow$ 1
- high $\rightarrow$ 2

Use with caution

This is appropriate for **ordered** categories, but not always for unordered categories because the numbers suggest an artificial order.

One-hot encoding

Suppose the color can be red, blue, or green.

Color	Is_red	Is_blue	Is_green
red	1	0	0
blue	0	1	0
green	0	0	1

Advantage

No false numerical order is introduced.

One-hot encoding with pandas: `get_dummies()`

```
1 encoded_df = pd.get_dummies(df, columns=['City'])
```

Result

One categorical column is replaced by several binary columns.

Train-test split: the key idea

- The training set is used to learn patterns.
- The test set is used to evaluate the model on unseen data.
- The test set must simulate future or new observations.

Why this matters

If we evaluate on data already used during training, the result is too optimistic.

Data leakage: a very important warning

- Data leakage happens when information from the test set enters the training process.
- Example of bad practice:
 - ▶ compute the global mean on the full dataset,
 - ▶ use it for imputation before train-test split.

Correct principle

Split the data first, then learn preprocessing parameters on the training set, and finally apply them to the test set.

A complete preprocessing pipeline

- 1 Inspect the dataset.
- 2 Correct data types.
- 3 Standardize text categories.
- 4 Handle missing values.
- 5 Remove true duplicates.
- 6 Check suspicious values and outliers.
- 7 Encode categorical variables.
- 8 Scale numerical variables when needed.
- 9 Split into training and test sets.
- 10 Train the model.

Table of contents

- 1 Common data quality problems
- 2 Data types and format consistency
- 3 Missing values
- 4 Duplicates and inconsistent records
- 5 Outliers and suspicious values
- 6 Preprocessing for machine learning
- 7 Summary**

Takeaways

- Real-world data is often incomplete, inconsistent, or noisy.
- Missing values can be deleted or imputed, depending on the context.
- Duplicates should be detected carefully, based on what one row represents.
- Outliers must be investigated, not removed blindly.
- Correct data types and consistent categories are essential.
- Before machine learning, we often need encoding and scaling.
- Preprocessing must avoid data leakage.

Final message

A good model starts with good data preparation.

In many projects, the quality of the preprocessing is as important as the choice of the model.

Preview of the next lecture

CM-3: Exploratory Data Analysis

Main question

How can we understand the structure and patterns of a dataset before modeling?

Topics:

- summarizing numerical and categorical variables;
- using descriptive statistics;
- visualizing distributions;
- studying relationships between variables;
- forming initial hypotheses from data.